

# Artificial Intelligence and Data Engineering for Orbital and Space Observation Systems

## Laboratory 7: First Visual AI Pipeline for Satellite Image Classification

Leszek Siwik  
Faculty of Space Technologies  
AGH University of Krakow

### 1 Introduction

In previous laboratories, you worked with tabular data stored in CSV files.

In this laboratory, you will work with images.

This is an important step because many real space observation systems process visual data such as:

- satellite imagery,
- land cover classification,
- Earth observation products.

So, today, you will build a simple end-to-end visual AI pipeline:

```
download dataset -> prepare images -> extract features  
-> train model -> predict -> visualize results
```

### 2 Dataset Preview

Before starting the laboratory, take a few minutes to explore the dataset manually.

In this lab, we use the EuroSAT RGB dataset based on Sentinel-2 satellite imagery.

You can preview the dataset here:

```
https://www.kaggle.com/datasets/apollo2506/eurosat-dataset
```

You do not need to download the dataset manually from Kaggle. It will be downloaded automatically in Task 1, but now just browse the images focusing on Forests, Rivers and Residentials. While browsing pay attention to: colors, textures, shapes, visible patterns, differences between classes and try to answer:

- Which class is the easiest to recognize?
- Which classes look similar?
- Which cases might be difficult for a model?

This short exploration will help you better understand the behavior of your model later.

### 3 Environment Setup

Install required libraries.

On Linux/macOS:

```
python3 -m pip install torch torchvision pillow numpy scikit-learn matplotlib joblib
```

All commands in this laboratory should be executed from the root directory of your project.

### 4 Project Structure

The targeted project structure is as follows

```
data/  
  raw/  
  processed/  
  inference_samples/  
  
models/  
reports/  
  
src/  
  data/  
    download_eurosat.py  
    prepare_image_dataset.py  
    create_noise_image.py  
  
  vision/  
    feature_extractor.py  
    train_image_model.py  
    predict_image.py
```

but we will create that step by step by completing the following tasks.

### 5 Task 1: Download Dataset

Create:

```
src/data/download_eurosat.py
```

Add the following code:

```
from pathlib import Path  
from torchvision.datasets import EuroSAT  
  
RAW_DATA_DIR = Path("data/raw")  
  
def main():  
    RAW_DATA_DIR.mkdir(parents=True, exist_ok=True)  
  
    dataset = EuroSAT(root=str(RAW_DATA_DIR), download=True)  
  
    print(f"Downloaded dataset with {len(dataset)} images")  
    print(f"Classes: {dataset.classes}")  
  
if __name__ == "__main__":
```

```
main()
```

Run:

```
python3 src/data/download_eurosat.py
```

or, if you use Python modules:

```
python3 -m src.data.download_eurosat
```

## Expected Output

You should see something similar to:

```
Downloaded dataset with 27000 images  
Classes: ['AnnualCrop', 'Forest', 'HerbaceousVegetation', ...]
```

## 6 Task 2: Prepare Dataset

The original EuroSAT dataset contains many classes.

For this laboratory, we will use only three:

```
Forest  
River  
Residential
```

The goal is to create the following structure:

```
data/processed/images/  
  train/  
    forest/  
    river/  
    residential/  
  test/  
    forest/  
    river/  
    residential/
```

Each class should contain:

- 48 training images,
- 12 testing images.

This gives 60 images per class in total.

### Task 2.1: Create the Dataset Preparation Script

Create:

```
src/data/prepare_image_dataset.py
```

At the beginning of the script, import the required libraries:

```
from pathlib import Path  
from PIL import Image
```

```
import random
import shutil
```

## Task 2.2: Define Configuration

Add the following configuration:

```
RAW_ROOT = Path("data/raw")
PROCESSED_ROOT = Path("data/processed/images")

SELECTED_CLASSES = {
    "Forest": "forest",
    "River": "river",
    "Residential": "residential"
}

IMAGES_PER_CLASS = 60
TRAIN_RATIO = 0.8
TARGET_SIZE = (64, 64)
RANDOM_SEED = 42

SUPPORTED_EXTENSIONS = [".jpg", ".jpeg", ".png"]
```

This means that:

- RAW\_ROOT points to the downloaded dataset,
- PROCESSED\_ROOT points to the new cleaned dataset,
- SELECTED\_CLASSES maps original EuroSAT names to lowercase folder names,
- IMAGES\_PER\_CLASS limits the dataset size,
- TARGET\_SIZE ensures that all images have the same size.

## Task 2.3: Find the EuroSAT Folder

Depending on the library version and operating system, the downloaded dataset may be stored in slightly different folders.

Therefore, do not hardcode the exact path.

Instead, create a function that searches for the folder containing the selected classes.

```
def find_eurosat_folder(raw_root):
    for path in raw_root.rglob("*"):
        if path.is_dir():
            class_names = [
                child.name for child in path.iterdir()
                if child.is_dir()
            ]

            if all(class_name in class_names for class_name in SELECTED_CLASSES.keys()):
                return path

    print("Error: Could not find EuroSAT class folders.")
    print("Make sure you ran src/data/download_eurosat.py first.")
    raise SystemExit(1)
```

## Task 2.4: Prepare Output Folders

Before creating a new processed dataset, remove the old one if it exists.

Then create the required train/test folders.

```
def prepare_output_directories():
    if PROCESSED_ROOT.exists():
        shutil.rmtree(PROCESSED_ROOT)

    for split_name in ["train", "test"]:
        for output_class in SELECTED_CLASSES.values():
            output_dir = PROCESSED_ROOT / split_name / output_class
            output_dir.mkdir(parents=True, exist_ok=True)
```

## Task 2.5: Process One Image

Create a helper function that:

- opens one image,
- converts it to RGB,
- resizes it to 64x64,
- saves it in the processed dataset folder.

```
def process_single_image(input_path, output_path):
    with Image.open(input_path) as image:
        image = image.convert("RGB")
        image = image.resize(TARGET_SIZE)
        image.save(output_path)
```

## Task 2.6: Prepare the Full Dataset

Now create the main dataset preparation function.

It should:

- find the EuroSAT folder,
- create output directories,
- iterate through selected classes,
- find image files,
- shuffle images,
- select 60 images per class,
- split them into train/test subsets,
- resize and save each image.

```
def prepare_dataset():
    random.seed(RANDOM_SEED)

    eurosat_folder = find_eurosat_folder(RAW_ROOT)
```

```

prepare_output_directories()

print("=== Preparing Image Dataset ===")
print(f"EuroSAT folder: {eurosat_folder}")
print(f"Processed dataset: {PROCESSED_ROOT}")

for original_class, output_class in SELECTED_CLASSES.items():
    class_dir = eurosat_folder / original_class

    image_files = [
        path for path in class_dir.iterdir()
        if path.suffix.lower() in SUPPORTED_EXTENSIONS
    ]

    random.shuffle(image_files)
    selected_files = image_files[:IMAGES_PER_CLASS]

    split_index = int(TRAIN_RATIO * len(selected_files))

    train_files = selected_files[:split_index]
    test_files = selected_files[split_index:]

    print(f"\nClass: {original_class}")
    print(f"Selected images: {len(selected_files)}")
    print(f"Training images: {len(train_files)}")
    print(f"Testing images: {len(test_files)}")

    for split_name, files in [("train", train_files), ("test", test_files)]:
        for index, input_path in enumerate(files):
            output_filename = f"{output_class}_{index:04d}.jpg"
            output_path = (
                PROCESSED_ROOT
                / split_name
                / output_class
                / output_filename
            )

            process_single_image(input_path, output_path)

```

## Task 2.7: Add the Main Function

At the end of the script, add:

```

def main():
    prepare_dataset()

if __name__ == "__main__":
    main()

```

## Task 2.8: Run the Script

Run:

```
python3 src/data/prepare_image_dataset.py
```

or:

```
python3 -m src.data.prepare_image_dataset
```

## Expected Output

You should see output similar to:

```
=== Preparing Image Dataset ===
EuroSAT folder: data/raw/eurosat/2750
Processed dataset: data/processed/images

Class: Forest
Selected images: 60
Training images: 48
Testing images: 12

Class: River
Selected images: 60
Training images: 48
Testing images: 12

Class: Residential
Selected images: 60
Training images: 48
Testing images: 12
```

## Task 2.9: Verify the Result

Check the generated folders:

```
ls data/processed/images/train/forest
ls data/processed/images/train/river
ls data/processed/images/train/residential

ls data/processed/images/test/forest
ls data/processed/images/test/river
ls data/processed/images/test/residential
```

Each training folder should contain 48 images.

Each testing folder should contain 12 images.

## Important Notes

- The processed dataset is intentionally small.
- This keeps the laboratory fast and easy to run.
- The original dataset remains unchanged in `data/raw/`.
- The processed dataset is recreated every time you run this script.
- If you want to start again, just rerun the script.

## 7 Task 3: Feature Extraction

Create:

```
src/vision/feature_extractor.py
```

Add:

```
import numpy as np

def extract_features(image):
    image = image.convert("RGB")
    image = image.resize((64, 64))

    array = np.array(image) / 255.0

    return array.flatten()
```

### What Happens Here?

An image is converted into a numerical vector.

For a 64x64 RGB image:

```
64 * 64 * 3 = 12288
```

So each image becomes one vector of 12288 numbers.

## 8 Task 4: Load Image Data

In this task, you will load the processed image dataset and convert it into two structures used by machine learning models:

- $X$  — feature vectors extracted from images,
- $y$  — class labels.

The model will not receive image files directly.

Instead, each image will be opened, converted into numerical features, and stored as one row in  $X$ .

### Task 4.1: Create the Training Script

Create:

```
src/vision/train_image_model.py
```

At the beginning of the script, import the required libraries:

```
from pathlib import Path
from PIL import Image
import numpy as np

from src.vision.feature_extractor import extract_features
```

Define the dataset path:

```
DATASET_DIR = Path("data/processed/images")
```

### Task 4.2: Load One Dataset Split

Create a function:



```
def load_image_split(split_dir):  
    ...
```

This function will load either the training split or the test split.  
For example:

```
data/processed/images/train/
```

or:

```
data/processed/images/test/
```

Each subfolder name is treated as the class label.

### Task 4.3: Implement the Loading Function

Complete function:

```
def load_image_split(split_dir):  
    X = []  
    y = []  
  
    class_dirs = sorted([  
        path for path in split_dir.iterdir()  
        if path.is_dir()  
    ])  
  
    for class_dir in class_dirs:  
        class_name = class_dir.name  
  
        image_files = sorted([  
            path for path in class_dir.iterdir()  
            if path.suffix.lower() in [".jpg", ".jpeg", ".png"]  
        ])  
  
        for image_path in image_files:  
            with Image.open(image_path) as image:  
                features = extract_features(image)  
  
                X.append(features)  
                y.append(class_name)  
  
    X = np.array(X)  
    y = np.array(y)  
  
    return X, y
```

### Task 4.4: Load Training and Test Data

Create another function:

```
def load_training_and_test_data():  
    train_dir = DATASET_DIR / "train"  
    test_dir = DATASET_DIR / "test"  
  
    X_train, y_train = load_image_split(train_dir)  
    X_test, y_test = load_image_split(test_dir)
```

```

print("=== Image ML Dataset ===")
print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")

return X_train, X_test, y_train, y_test

```

### Task 4.5: Add Temporary Main Function

At this stage, the script should only verify that image loading works.

Add:

```

def main():
    X_train, X_test, y_train, y_test = load_training_and_test_data()

if __name__ == "__main__":
    main()

```

### Task 4.6: Run the Script

Because the script imports from `src`, run it from the project root as a module:

```
python3 -m src.vision.train_image_model
```

### Expected Output

You should see output similar to:

```

=== Image ML Dataset ===
X_train shape: (144, 12288)
y_train shape: (144,)
X_test shape: (36, 12288)
y_test shape: (36,)

```

The number 12288 means:

```
64 * 64 * 3 = 12288
```

## 9 Task 5: Train Model

In this task, you will train your first image classification model.

The model will learn to assign one of the following labels to an image:

```

forest
river
residential

```

### Task 5.1: Import the Classifier

At the top of:

```
src/vision/train_image_model.py
```

add:

```
from sklearn.ensemble import RandomForestClassifier
```

## Task 5.2: Create the Training Function

Create:

```
def train_model(X_train, y_train):
    print("=== Training Image Classifier ===")
    print(f"Training samples: {len(X_train)}")
    print(f"Number of features per image: {X_train.shape[1]}")

    model = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )

    model.fit(X_train, y_train)

    print("Model trained.")

    return model
```

## Task 5.3: Update the Main Function

Update your temporary main() function:

```
def main():
    X_train, X_test, y_train, y_test = load_training_and_test_data()

    model = train_model(X_train, y_train)

if __name__ == "__main__":
    main()
```

## Task 5.4: Run the Script

Run:

```
python3 -m src.vision.train_image_model
```

# 10 Task 6: Evaluate Model

In this task, you will evaluate how well your trained model performs on unseen data.

The evaluation will be performed on:

```
X_test
y_test
```

## Task 6.1: Import Required Function

At the top of:

```
src/vision/train_image_model.py
```

add:

```
from sklearn.metrics import accuracy_score
```

## Task 6.2: Create Evaluation Function

Add:

```
def evaluate_model(model, X_test, y_test):
    print("=== Model Evaluation ===")

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    print(f"Number of test samples: {len(X_test)}")
    print(f"Accuracy: {accuracy:.4f}")

    return accuracy
```

## Task 6.3: Update Main Function

Update:

```
def main():
    X_train, X_test, y_train, y_test = load_training_and_test_data()

    model = train_model(X_train, y_train)

    evaluate_model(model, X_test, y_test)

if __name__ == "__main__":
    main()
```

## Expected Output

You should see something similar to:

```
=== Model Evaluation ===
Number of test samples: 36
Accuracy: 0.8056
```

The exact value may differ.

## 11 Task 7: Save Model

Without saving the model, you would need to retrain it every time.

Saving the model allows you to:

- reuse the trained model later,
- use it later for inference without retraining.

### Task 7.1: Import Required Library

At the top of:

```
src/vision/train_image_model.py
```

add:

```
import joblib
```

### Task 7.2: Define Model Path

Define where the model will be saved:

```
MODEL_PATH = Path("models/image_model.joblib")
```

### Task 7.3: Create Save Function

Add:

```
def save_model(model):
    MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)

    joblib.dump(model, MODEL_PATH)

    print("=== Saving Model ===")
    print(f"Saved model: {MODEL_PATH}")
```

### Task 7.4: Update Main Function

Update:

```
def main():
    X_train, X_test, y_train, y_test = load_training_and_test_data()

    model = train_model(X_train, y_train)

    evaluate_model(model, X_test, y_test)

    save_model(model)

if __name__ == "__main__":
    main()
```

### Task 7.5: Verify the File

Check:

```
ls models/
```

You should see:

```
image_model.joblib
```

## 12 Task 8: Predict Single Image

In this task, you will use your trained model to classify a single image.

This is the first step from:

```
training
```

to:

```
real usage (inference)
```

### Task 8.1: Create Script

Create:

```
src/vision/predict_image.py
```

### Task 8.2: Import Required Libraries

At the top of the file, add:

```
from pathlib import Path
from PIL import Image
import joblib

from src.vision.feature_extractor import extract_features
```

### Task 8.3: Define Path

Add:

```
MODEL_PATH = Path("models/image_model.joblib")
```

### Task 8.4: Load the Model

Add:

```
def load_model():
    if not MODEL_PATH.exists():
        print(f"Error: model file not found: {MODEL_PATH}")
        raise SystemExit(1)

    model = joblib.load(MODEL_PATH)

    print("Model loaded.")

    return model
```

### Task 8.5: Predict One Image

Add:

```
def predict_image(model, image_path):
    path = Path(image_path)

    if not path.exists():
```

```

        print(f"Error: file not found: {image_path}")
        return

    with Image.open(path) as image:
        features = extract_features(image)

    prediction = model.predict([features])[0]

    print("=== Prediction ===")
    print(f"Image: {image_path}")
    print(f"Predicted class: {prediction}")

```

## Task 8.6: Main Function

Add:

```

def main():
    model = load_model()

    image_path = "data/processed/images/test/forest/forest_0000.jpg"

    predict_image(model, image_path)

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.predict_image
```

## Expected Output

You should see output similar to:

```

Model loaded.

=== Prediction ===
Image: data/processed/images/test/forest/forest_0000.jpg
Predicted class: forest

```

Please, note, that at this stage, the goal is only to verify that the saved model can be loaded and used for inference on one image. More visual tests will be performed in the next task.

## 13 Task 9: Visualize Prediction

In this task, you will display the image together with the predicted label.

This allows you to visually verify whether the model prediction makes sense.

### Task 9.1: Import Matplotlib

At the top of:

```
src/vision/predict_image.py
```

add:

```
import matplotlib.pyplot as plt
```

### Task 9.2: Modify Prediction Function

Replace your `predict_image` function with:

```
def predict_image(model, image_path):
    path = Path(image_path)

    if not path.exists():
        print(f"Error: file not found: {image_path}")
        return

    with Image.open(path) as image:
        features = extract_features(image)
        image_for_plot = image.copy()

    prediction = model.predict([features])[0]

    print("=== Prediction ===")
    print(f"Image: {image_path}")
    print(f"Predicted class: {prediction}")

    plt.imshow(image_for_plot)
    plt.title(f"Prediction: {prediction}")
    plt.axis("off")
    plt.show()
```

### Task 9.3: Run

Run:

```
python -m src.vision.predict_image
```

A window should appear displaying the image and predicted class.

## 14 Task 10: Additional Experiments and Limitations

So far, you tested images from known classes.

Now check how the model behaves on inputs outside its normal training domain.

### Experiment 1: Image from an Unknown EuroSAT Class

Use a class that was not used during training, for example:

```
data/raw/eurosat/2750/Highway/Highway_1.jpg
```

Modify:

```
image_path = "data/raw/eurosat/2750/Highway/Highway_1.jpg"
```

Run:

```
python -m src.vision.predict_image
```

Observe:



- What class is predicted?
- Does the prediction make sense?

## Experiment 2: Noise Image

Create:

```
src/data/create_noise_image.py
```

Add:

```
from pathlib import Path
import numpy as np
from PIL import Image

output_path = Path("data/inference_samples/noise.jpg")
output_path.parent.mkdir(parents=True, exist_ok=True)

array = np.random.randint(0, 256, size=(64, 64, 3), dtype=np.uint8)
image = Image.fromarray(array)

image.save(output_path)

print(f"Saved noise image: {output_path}")
```

Run:

```
python -m src.data.create_noise_image
```

Then test:

```
image_path = "data/inference_samples/noise.jpg"
```

Run prediction again. What the result/answer are you getting?

## 15 Task 11: Independent Extension — Compare Multiple Models

So far, you used one model:

```
RandomForestClassifier
```

Now you will compare several models and visualize how they classify the same image.

Also, the goal is to answer two questions:

- Which model achieves the best accuracy?
- Do different models classify the same image in the same way?

You will also compare model performance with training time.

### Task 11.1: Add More Models

Extend your training script with at least three models.

Use, for example:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
```

Create a dictionary of models:

```
models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    "KNN": KNeighborsClassifier(
        n_neighbors=3
    ),
    "Logistic Regression": LogisticRegression(
        max_iter=1000
    ),
    "SVM": SVC()
}
```

### Task 11.2: Measure Training Time

Use the `time` module:

```
import time
```

For each model, measure how long training takes:

```
start_time = time.time()

model.fit(X_train, y_train)

end_time = time.time()
training_time = end_time - start_time
```

### Task 11.3: Evaluate Each Model

For each trained model:

- generate predictions on `X_test`,
- compute accuracy,
- save model name, accuracy, and training time.

Example result format:

```
results.append({
    "model_name": model_name,
    "accuracy": accuracy,
    "training_time": training_time,
    "model": model
})
```

Print results:

```
Model: Random Forest
Accuracy: 0.8333
Training time: 1.24 s
```

### Task 11.4: Create Accuracy vs Training Time Plot

Create a plot showing the relationship between model accuracy and training time.

### Task 11.5: Visualize Predictions from All Models

Extend your prediction script so that one image is classified by all trained models.

For a selected image, display:

```
Image: data/processed/images/test/forest/forest_0000.jpg

Random Forest: forest
KNN: forest
Logistic Regression: river
SVM: forest
```

The result should also be shown visually on the image.

Example title:

```
Random Forest: forest | KNN: forest | Logistic Regression: river | SVM: forest
```

## Important Notes

- Use the same training and test data for all models.
- Use the same feature extraction method for all models.
- Do not change image size or preprocessing.
- The goal is fair comparison, not optimization.

## 16 Submission

Submit the following artifacts:

- dataset download script,
- dataset preparation script,
- feature extractor,
- training script,
- trained model or models, depending on your Task 11 implementation,
- prediction and visualization script,
- noise image script, if completed,
- model comparison plot,
- model comparison summary,
- short reflection answering the following questions:

- How is an image converted into numbers?
  - What does the model actually “see”?
  - Why can the model classify random or unknown images as one of the known classes?
  - Why can this behavior be dangerous in real AI systems?
  - What would need to be improved before using such a system operationally?
  - Which model achieved the highest accuracy?
  - Which model was fastest to train?
  - Did the fastest model also achieve the best accuracy?
  - Did all models classify the same image in the same way?
  - What does the accuracy vs training time plot show?
- repository link.